

Listas Ligadas Lineares

Prof. Denilson Fagundes Barbosa

Instituto Federal de Santa Catarina
Câmpus Canoinhas

2026

Objetivos da aula

- Identificar as desvantagens do armazenamento sequencial para pilhas e filas.
- Compreender a estrutura e o funcionamento de listas ligadas lineares.
- Aprender os algoritmos de inserção e remoção de nós em listas ligadas.
- Diferenciar a implementação de listas ligadas usando vetores e alocação dinâmica em C.
- Implementar pilhas, filas e filas de prioridade utilizando a estrutura de listas ligadas.
- Entender a utilidade e a implementação de nós de cabeçalho em listas.
- Analisar a eficiência de tempo e espaço entre diferentes métodos de implementação de listas.

1 Listas Ligadas Lineares

2 Listas em C

1 Listas Ligadas Lineares

2 Listas em C

Limitações do Armazenamento Sequencial

- **Alocação Fixa:** Uma quantidade fixa de memória permanece alocada para pilhas ou filas, mesmo quando a estrutura está vazia.
- **Risco de Estouro:** Não é possível alocar mais do que a quantidade fixa inicial, limitando o crescimento da estrutura.
- **Inflexibilidade:** Em sistemas com múltiplas estruturas, uma pilha pode atingir seu limite enquanto outra possui espaço ocioso considerável.
- **Custo de Movimentação:** Inserções ou remoções no meio de um vetor exigem o deslocamento físico de diversos elementos.

Compartilhamento de Espaço em Vetores

- **Solução para Duas Pilhas:** É possível alocar um único vetor para duas pilhas, onde a primeira cresce a partir do índice 0 e a segunda a partir do final do vetor.
- **Eficiência:** Uma pilha pode utilizar o espaço não ocupado pela outra, otimizando o uso da memória total disponível.
- **Limitação Crítica:** Este esquema não se expande de forma simples para três ou mais pilhas, nem para o uso de filas.
- **Complexidade de Gerenciamento:** Para múltiplas estruturas, seria necessário rastrear inícios e fins, exigindo deslocamentos constantes de estruturas vizinhas.

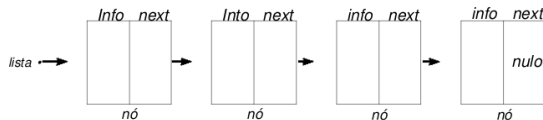
Conceito de Lista Ligada Linear

- **Ordenação Explícita:** Diferente do vetor (ordem implícita), cada item em uma lista ligada contém o endereço do próximo elemento.
- **Nó (Node):** Unidade básica composta por dois campos: informação (*info*) e endereço do próximo (*next*).
- **Ponteiro:** O campo de endereço que permite acessar o próximo nó da sequência.
- **Ponteiro Externo:** Variável que armazena o endereço do primeiro nó da lista, servindo como ponto de acesso inicial.
- **Valor Null:** Valor especial usado no campo *next* do último nó para indicar o fim da lista.

Estrutura de uma Lista Ligada Linear

Representação visual de uma lista ligada linear.

- **lista:** Ponteiro externo para o início.
- **nó:** Contém os campos *info* e *next*.
- **nulo:** Indica o encerramento da estrutura.



Uma lista ligada linear

Diagrama mostrando quatro retângulos (nós) conectados por setas. O primeiro é apontado por uma seta externa 'lista'. O último contém a palavra 'nulo'. Cada nó é dividido em 'info' e 'next'.

Para a descrição de algoritmos, utiliza-se a seguinte notação (onde p é um ponteiro):

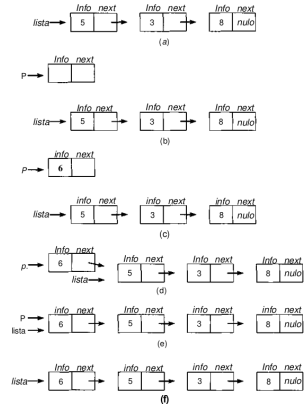
- $node(p)$: Refere-se ao nó apontado por p .
- $info(p)$: Refere-se à parte de informação do nó $node(p)$.
- $next(p)$: Refere-se ao campo de endereço seguinte (ponteiro) do nó $node(p)$.
- **Acesso Sequencial:** Se $next(p)$ não for nulo, $info(next(p))$ refere-se à informação do nó sucessor.

- **Estrutura Dinâmica:** O número de nós pode variar consideravelmente durante a execução, conforme elementos são inseridos ou removidos.
- **Contraste com Vetores:** Vetores possuem natureza estática, com tamanho constante definido em tempo de compilação ou alocação inicial.
- **Mecanismo de Alocação:** Diferente do vetor, a lista não possui locais pré-fornecidos; é necessário um mecanismo para obter novos nós vazios.
- **Operação `getnode()`:** Função abstrata que obtém um nó vazio e retorna seu endereço para uma variável ponteiro.

Inclusão no Início da Lista

Processo passo a passo para incluir o valor 6 no início de uma lista.

- **(a)-(b):** Obtenção de um novo nó via *getnode*.
- **(c):** Atribuição do valor ao campo *info*.
- **(d):** Ajuste do ponteiro *next* para o antigo início.
- **(e)-(f):** Atualização do ponteiro externo *lista*.



Incluindo um elemento no início

Sequência de seis diagramas (a a f) demonstrando a evolução dos ponteiros e a criação de um novo nó para inserir o valor 6 antes do valor 5 em uma lista existente.

Algoritmo de Inclusão no Início

Para incluir um objeto x no início de uma lista $list$:

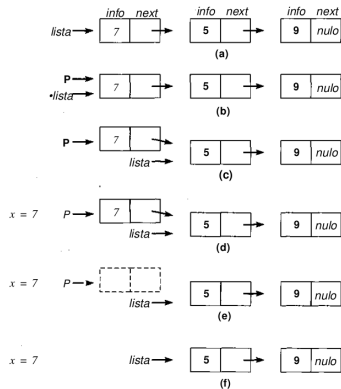
```
1 p = getnode();  
2 info(p) = x;  
3 next(p) = list;  
4 list = p;  
5
```

- O algoritmo funciona corretamente mesmo que a lista esteja inicialmente vazia ($list == null$).
- A variável p atua como um ponteiro auxiliar temporário durante o processo.

Remoção do Início da Lista

Processo de remoção do primeiro nó e armazenamento de seu valor em x .

- **(a)-(b):** Apontar ponteiro auxiliar p para o primeiro nó.
- **(c)-(d):** Avançar o ponteiro $lista$ para o próximo nó.
- **(e)-(f):** Liberar o nó removido para reutilização.



Removendo um nó do início

Série de diagramas mostrando a desconexão do primeiro nó (contendo o valor 7) de uma lista, a atualização do ponteiro 'lista' para o nó seguinte (valor 5) e a liberação do nó original.

Algoritmo de Remoção do Início

Para remover o primeiro elemento de uma lista não-vazia e armazenar seu valor em x :

```
1 p = list;  
2 list = next(p);  
3 x = info(p);  
4 freenode(p);  
5
```

- **freenode(p)**: Operação essencial para tornar o nó disponível para reutilização, evitando desperdício de memória.
- Sem o *freenode*, o nó torna-se inacessível (vazamento de memória), pois não há mais ponteiros apontando para ele.

- **Analogia:** A inclusão/remoção no início de uma lista é análoga às operações de uma pilha.
- **Topo da Pilha:** O primeiro nó da lista representa o topo.
- **Operação Push:** Implementada pela inserção no início da lista.
- **Operação Pop:** Implementada pela remoção do primeiro nó, com tratamento de *underflow* se a lista estiver vazia.
- **Operação Empty:** Verifica se o ponteiro externo é igual a *null*.

Implementação Ligada de Filas

- **Estrutura:** Uma fila q requer dois ponteiros externos: $q.front$ (início) e $q.rear$ (final).
- **Remoção ($removeq$):** Ocorre em $q.front$. Se for o último elemento, tanto $front$ quanto $rear$ devem tornar-se *null*.
- **Inserção ($insertq$):** Ocorre após o nó apontado por $q.rear$.
- **Vantagem de Espaço:** Embora cada nó use mais memória (devido ao ponteiro), o espaço total é compartilhado dinamicamente entre todas as filas e pilhas do programa.

Inserção (insert(q, x)):

```
1 p = getnode(); info(p) = x; next(p) = null;  
2 if (q.rear == null) q.front = p;  
3 else next(q.rear) = p;  
4 q.rear = p;  
5
```

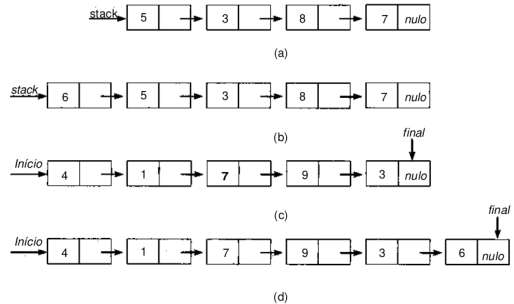
Remoção (remove(q)):

```
1 p = q.front; x = info(p);  
2 q.front = next(p);  
3 if (q.front == null) q.rear = null;  
4 freenode(p); return(x);  
5
```

Pilhas e Filas como Listas Ligadas

Comparação visual de estruturas lineares.

- **(a)-(b):** Pilha (*stack*) antes e depois de um *push*.
- **(c)-(d):** Fila (*queue*) antes e depois de um *insert* no final.
- **Vantagem:** Múltiplas estruturas podem compartilhar o mesmo conjunto de nós disponíveis.



Pilha e fila como listas ligadas

Diagramas de listas ligadas. A pilha mostra inserção no topo (esquerda). A fila mostra ponteiros para 'Início' e 'final', com inserção ocorrendo após o último nó.

As Operações *getnode* e *freenode*

- **Mundo Ideal:** Em um sistema com memória infinita, *freenode* seria desnecessário, pois sempre haveria nós novos.
- **Mundo Real:** A memória é finita. Nós que não estão mais em uso devem ser devolvidos a um reservatório para reutilização.
- **Lista Livre (Available List):** O reservatório de nós vazios é, ele próprio, uma lista ligada atuando como uma pilha.
- **Funcionamento:**
 - *getnode*: Remove o primeiro nó da lista livre.
 - *freenode*: Insere o nó liberado no início da lista livre.

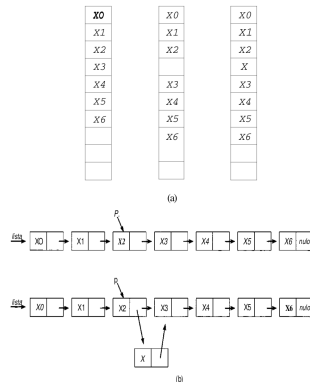
Listas Ligadas como Estruturas de Dados

- **Acesso Aleatório:** Desvantagem em relação a vetores. Para acessar o n -ésimo item, é necessário percorrer $n - 1$ elementos anteriores.
- **Inserção/Remoção no Meio:** Grande vantagem. Não requer deslocamento de outros elementos, apenas o ajuste de ponteiros.
- **Flexibilidade:** Permite a criação de estruturas complexas e não-homogêneas.
- **Custo de Memória:** O campo *next* representa um *overhead*. Contudo, em estruturas com muitos campos de dados, esse acréscimo percentual é pequeno.

Inserção em Vetor vs. Lista Ligada

Comparação da inserção de um elemento X entre X_2 e X_3 .

- **Vetor (a):** Exige deslocar X_3, X_4, X_5, X_6 para abrir espaço.
- **Lista (b):** Requer apenas a criação de um nó e o ajuste de dois ponteiros.



Inserção no meio da estrutura

A parte (a) mostra um vetor sofrendo deslocamento de elementos. A parte (b) mostra uma lista ligada onde um novo nó é inserido entre dois existentes apenas redirecionando as setas de conexão.

Inserir após o nó p (insertafter(p, x)):

```
1 q = getnode();  
2 info(q) = x;  
3 next(q) = next(p);  
4 next(p) = q;  
5
```

Remover após o nó p (delafter(p, x)):

```
1 q = next(p);  
2 x = info(q);  
3 next(p) = next(q);  
4 freenode(q);  
5
```

Nota: Não é possível inserir antes de um nó ou remover o próprio nó p sem conhecer seu predecessor.

- **Lista Ordenada:** Os itens são mantidos em uma sequência específica (ex: ordem ascendente).
- **Operação `place(list, x)`:** Insere um elemento x em sua posição correta.
- **Análise de Desempenho:**
 - Para uma lista com n nós, a probabilidade de inserção em qualquer posição é $1/(n + 1)$.
 - O número médio de acessos a nós para uma inserção aleatória é aproximadamente $n/2$.

- **Lista Ordenada:**

- *pqinsert*: Usa a operação *place*.
- *pqmindelete*: Remove o primeiro elemento.

- **Lista Desordenada:**

- *pqinsert*: Insere no início ou fim.
- *pqmindelete*: Requer percorrer a lista inteira para encontrar o menor elemento.

- **Vantagem sobre Vetores:** Não há necessidade de deslocamento físico de elementos durante a manutenção da ordem.

Nós de Cabeçalho (Header Nodes)

- **Definição:** Um nó adicional mantido no início da lista que não representa um item de dados.
- **Utilidade:**
 - Armazenar informações globais (ex: contagem de nós).
 - Simplificar algoritmos, garantindo que a lista nunca esteja tecnicamente "vazia" (sempre há o cabeçalho).
 - Facilitar a implementação de filas com um único ponteiro externo.
- **Lista Vazia:** Representada por uma lista contendo apenas o nó de cabeçalho com ponteiro *next* para *null*.

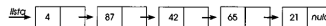
Exemplos de Nós de Cabeçalho

Diferentes configurações de cabeçalhos.

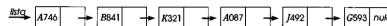
- **(a)**: Cabeçalho simples sem uso de *info*.
- **(b)**: Campo *info* armazena o número de nós.
- **(c)**: Cabeçalho representando uma montagem industrial.
- **(e)**: Cabeçalho com ponteiro para o último nó.



(a)



(b)



(c)



(d)



(e)

Listas com nós de cabeçalho

Cinco variações de listas ligadas (a a e) demonstrando o uso do primeiro nó como cabeçalho para metadados ou simplificação de acesso ao final da lista.

Roteiro da aula

1 Listas Ligadas Lineares

2 Listas em C

Implementação de Listas em Vetor

- Uma lista linear pode ser representada em C como um conjunto de nós organizados em um vetor.
- Diferente de um vetor comum, a ordem dos elementos não é dada pelos índices, mas por ponteiros explícitos.
- Cada nó deve conter o dado (informação) e um ponteiro (índice) para o seu sucessor.
- Um grupo de nós pode ser declarado como uma estrutura:

Estrutura de Nó em Vetor

```
struct nodetype { int info, next; };  
struct nodetype node[NUMNODES];
```

Ponteiros e Representação de Índices

- Nesta implementação, um “ponteiro” é um número inteiro entre 0 e $NUMNODES - 1$.
- O valor -1 é utilizado para representar o ponteiro nulo (`null`).
- Se p é o índice de um nó, `node[p].info` acessa o dado e `node[p].next` acessa o índice do próximo elemento.
- Exemplo: Se `list = 7`, o primeiro nó está em `node[7]`. Se `node[7].next = 385`, o segundo nó está em `node[385]`.
- A variável global `avail` aponta para o início de uma lista de nós livres (disponíveis para uso).

Vetor de Nós com Múltiplas Listas

Representação de um vetor node contendo quatro listas ligadas distintas.

- **list1:** Inicia no índice 16.
- **list2:** Inicia no índice 4.
- **list3:** Inicia no índice 11.
- **list4:** Inicia no índice 3.
- **Encadeamento:** O campo next direciona para o próximo índice ou -1 para o fim.

	info	next
0	26	-1
1	11	9
2	5	15
lista 4 = 3	1	24
lista 2 = 4	17	0
5	13	1
6		
7	19	18
8	14	12
9	4	21
10		
lista 3 = 11	31	7
12	6	2
13		
14		
15	37	23
lista 1 = 16	3	20
17		
18	32	-1
19		
20	7	8
21	15	-1
22		
23	12	-1
24	18	5
25		
26		

Vetor de nós com quatro listas

A imagem mostra uma tabela representando um vetor de memória com colunas para índice, info e next. Diversas setas indicam o encadeamento lógico entre índices não contíguos, formando quatro listas separadas.

Operações getnode e freenode em Vetor

- `getnode()`: Remove um nó da lista de disponíveis (`avail`) e retorna seu índice.
- `freenode(p)`: Devolve o nó de índice `p` para a lista `avail`.

```
1 int getnode(void) {  
2     int p;  
3     if (avail == -1) {  
4         printf("estouro\n");  
5         exit(1);  
6     }  
7     p = avail;  
8     avail = node[avail].next;  
9     return(p);  
10 }  
11  
12 void freenode(int p) {  
13     node[p].next = avail;  
14     avail = p;  
15 }  
16
```

Primitivas de Inserção e Remoção em Vetor

- `insafter(p, x)`: Insere o valor `x` após o nó de índice `p`.
- `delafter(p, px)`: Remove o nó após `p` e armazena seu valor em `*px`.

```
1 void insafter(int p, int x) {
2     int q;
3     if (p == -1) { printf("insercao nula\n"); return; }
4     q = getnode();
5     node[q].info = x;
6     node[q].next = node[p].next;
7     node[p].next = q;
8 }
9
10 void delafter(int p, int *px) {
11     int q;
12     if ((p == -1) || (node[p].next == -1)) {
13         printf("remocao nula\n"); return;
14     }
15     q = node[p].next;
16     *px = node[q].info;
17     node[p].next = node[q].next;
18     freenode(q);
19 }
20
```

Limitações da Implementação em Vetor

- **Tamanho Fixo:** O número de nós deve ser previsto no momento da escrita do programa, o que pode levar a desperdício de memória ou estouro.
- **Alocação Estática:** O espaço para o vetor permanece reservado durante toda a execução, mesmo que apenas uma pequena fração seja utilizada.
- **Solução:** Uso de nós dinâmicos, onde o armazenamento é reservado apenas quando necessário e liberado quando não for mais útil.

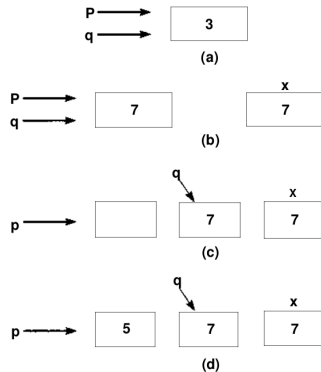
Alocação Dinâmica de Variáveis

- Em C, a função `malloc(size)` aloca dinamicamente uma parte da memória de tamanho `size`.
- O operador `sizeof` é usado para garantir a independência de máquina ao calcular o tamanho em bytes de um tipo.
- `pi = (int *) malloc(sizeof(int));` cria um inteiro dinâmico e armazena seu endereço em `pi`.
- A função `free(p)` libera o espaço ocupado pela variável apontada por `p`, tornando-o disponível para o sistema.

Visualização de Operações com Ponteiros

Evolução do estado da memória e ponteiros.

- **(a):** p e q apontam para o mesmo local contendo o valor 3.
- **(b):** O valor é alterado para 7; ambos refletem a mudança.
- **(c):** p recebe um novo endereço via `malloc`.
- **(d):** p é inicializado com 5, enquanto q permanece com 7.



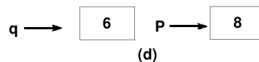
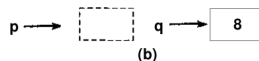
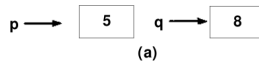
Manipulação dinâmica de ponteiros

Diagramas de blocos mostrando ponteiros p e q apontando para caixas de memória. As etapas mostram a atribuição de valores, compartilhamento de endereços e a criação de novas instâncias de memória.

Ponteiros Perdidos e Memória Inacessível

Riscos da má gestão de memória.

- **Vazamento de Memória:** Se `malloc` é chamado duas vezes para o mesmo ponteiro sem salvar o endereço anterior, a primeira cópia é perdida e não pode ser liberada.
- **Ponteiro Perdido (Dangling Pointer):** Após `free(p)`, o valor de `p` ainda é um endereço, mas o objeto naquele local não é mais válido.
- **Recomendação:** Definir `p = NULL` imediatamente após `free(p)`.



Efeitos da liberação de memória

A figura ilustra o estado de ponteiros antes e depois da função `free`. Mostra um nó sendo invalidado (tracejado) e a situação onde dois ponteiros apontavam para o mesmo local que foi liberado.

Listas Ligadas com Variáveis Dinâmicas

- Utiliza-se uma estrutura autorreferenciada para definir o nó.
- O campo `next` agora é um ponteiro real de memória, não um índice de vetor.

```
1 struct node {  
2     int info;  
3     struct node *next;  
4 };  
5 typedef struct node *NODEPTR;  
6  
7 NODEPTR getnode(void) {  
8     NODEPTR p;  
9     p = (NODEPTR) malloc(sizeof(struct node));  
10    return(p);  
11 }  
12  
13 void freenode(NODEPTR p) {  
14     free(p);  
15 }  
16
```

Primitivas Dinâmicas: insafter e delafter

```
1 void insafter(NODEPTR p, int x) {
2     NODEPTR q;
3     if (p == NULL) {
4         printf("inversao nula\n");
5         exit(1);
6     }
7     q = getnode();
8     q->info = x;
9     q->next = p->next;
10    p->next = q;
11 }
12
13 void delafter(NODEPTR p, int *px) {
14     NODEPTR q;
15     if ((p == NULL) || (p->next == NULL)) {
16         printf("remocao nula\n");
17         exit(1);
18     }
19     q = p->next;
20     *px = q->info;
21     p->next = q->next;
22     freenode(q);
23 }
24
```

- A lógica é idêntica à do vetor, alterando apenas a forma de referenciar os nós (setas -> em vez de colchetes []).

Filas como Listas em C

- Uma fila *q* consiste em uma lista e dois ponteiros: *front* (início) e *rear* (fim).

Implementação em Vetor

```
1 struct queue {  
2     int front, rear;  
3 };  
4 struct queue q;
```

Implementação Dinâmica

```
1 struct queue {  
2     NODEPTR front, rear;  
3 };  
4 struct queue q;  
5
```

- Na fila vazia, ambos os ponteiros devem ser nulos (-1 ou NULL).
- A função `empty(pq)` verifica se `pq->front` é nulo.

Operação de Inserção em Fila (insert)

Vetor:

```
1 void insert(struct queue *pq, int x) {  
2     int p;  
3     p = getnode();  
4     node[p].info = x;  
5     node[p].next = -1;  
6     if (pq->rear == -1)  
7         pq->front = p;  
8     else  
9         node[pq->rear].next = p;  
10    pq->rear = p;  
11 }  
12
```

Dinâmica:

```
1 void insert(struct queue *pq, int x) {  
2     NODEPTR p;  
3     p = getnode();  
4     p->info = x;  
5     p->next = NULL;  
6     if (pq->rear == NULL)  
7         pq->front = p;  
8     else  
9         (pq->rear)->next = p;  
10    pq->rear = p;  
11 }  
12
```

Operação de Remoção em Fila (remove)

- Atenção especial: ao remover o último elemento, rear também deve tornar-se nulo.

```
1 int remove(struct queue *pq) {  
2     NODEPTR p; int x;  
3     if (empty(pq)) {  
4         printf("underflow na fila\n");  
5         exit(1);  
6     }  
7     p = pq->front;  
8     x = p->info;  
9     pq->front = p->next;  
10    if (pq->front == NULL)  
11        pq->rear = NULL;  
12    freenode(p);  
13    return(x);  
14 }  
15
```

Exemplo: Inserção Ordenada (place)

- A função `place(plist, x)` insere `x` em uma lista classificada.
- Utiliza dois ponteiros auxiliares (`p` e `q`) para percorrer a lista.

```
1 void place(NODEPTR *plist, int x) {  
2     NODEPTR p, q;  
3     q = NULL;  
4     for (p = *plist; p != NULL && x > p->info; p = p->next)  
5         q = p;  
6     if (q == NULL)  
7         push(plist, x); // Insere no inicio  
8     else  
9         insafter(q, x); // Insere no meio ou fim  
10 }  
11
```

- **Nota:** `plist` é passado como ponteiro para ponteiro (`NODEPTR *`) pois o início da lista pode mudar.

Listas Não-Homogêneas

- Para representar listas que contêm diferentes tipos de dados (inteiros, reais, strings), utiliza-se uma `union`.
- Um campo `etype` identifica qual tipo de dado está armazenado no nó atual.

```
1 struct node {  
2     int etype; // INTGR, FLT ou STRING  
3     union {  
4         int ival;  
5         float fval;  
6         char *pval;  
7     } element;  
8     struct node *next;  
9 };  
10
```

- O `sizeof` de uma união é sempre grande o suficiente para o seu maior componente.

Comparação: Vetor vs. Dinâmica

- **Vantagem da Dinâmica:** Não reserva memória antecipadamente; o estouro só ocorre se a memória total do sistema acabar.
- **Vantagem do Vetor:** Mais rápido para criar/liberar nós (apenas manipulação de índices), enquanto a dinâmica depende do gerenciamento do sistema operacional (`malloc/free`).
- **Acesso:** Referenciar `*p` é direto; calcular `node[p]` exige somar o índice ao endereço base do vetor.

Nós de Cabeçalho (Header Nodes)

- Um nó adicional no início da lista que não representa um item de dado.
- Pode armazenar informações globais como o tamanho da lista ou um ponteiro para o último nó.
- Simplifica operações: uma lista vazia passa a ser uma lista com apenas o nó de cabeçalho (ponteiro externo nunca é NULL).

Exemplo de Cabeçalho para Strings

```
1 struct charstr {  
2     int length;  
3     struct node *firstchar;  
4 };  
5 struct charstr s1, s2;  
6
```

- Listas ligadas resolvem o problema de alocação fixa de memória, permitindo que estruturas de dados cresçam e diminuam dinamicamente conforme a necessidade.
- A implementação dinâmica em C, utilizando ponteiros e as funções malloc/free, é geralmente superior à implementação em vetor por não exigir reserva antecipada de espaço.
- Embora o acesso ao enésimo elemento seja mais lento em listas do que em vetores, a inserção e remoção de elementos no meio da estrutura são muito mais eficientes em listas ligadas.
- O uso de nós de cabeçalho pode simplificar a lógica de programação, evitando casos especiais para listas vazias e facilitando o rastreamento de metadados da lista.

- TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. **Estruturas de dados usando C**. São Paulo: Makron Books, 1995.